Summarize

Part I. Pulling Things Apart

As many of us discovered when we were children, a great way to understand how something fits together is to first pull it apart. To understand complex subjects (such as trade-offs in distributed architectures), an architect must figure out where to start untangling.

In the book *What Every Programmer Should Know About Object-Oriented Design* (Dorset House), Meilir Page-Jones made the astute observation that coupling in architecture may be split into static and dynamic coupling. *Static* coupling refers to the way architectural parts (classes, components, services, and so on) are *wired* together: dependencies, coupling degree, connection points, and so on. An architect can often measure static coupling at compile time as it represents the static dependencies within the architecture.

Dynamic coupling refers to how architecture parts *call* one another: what kind of communication, what information is passed, strictness of contracts, and so on.

Our goal is to investigate how to do trade-off analysis in distributed architectures; to do that, we must pull the moving pieces apart so that we can discuss them in isolation to understand them fully before putting them back together.

Part I primarily deals with *architectural structure*, how things are statically coupled together. In Chapter 2, we tackle the problem of defining the scope of static and dynamic coupling in architectures, and present the entire picture that we must pull apart to understand. Chapter 3 begins that process, defining modularity and separation in architecture. Chapter 4 provides tools to evaluate and deconstruct codebases, and Chapter 5 supplies patterns to assist the process.

Data and transactions have become increasingly important in architecture, driving many trade-off decisions by architects and DBAs. Chapter 6 addresses the architectural impacts of data, including how to reconcile service and data boundaries. Finally, Chapter 7 ties together architecture coupling with data concerns to define *integrators* and *disintegrators*—forces that encourage a larger or smaller service size and boundary.

Previous chapter

< 1. What Happens When There Are No “Best Practices”?

Next chapter

2. Discerning Coupling in Software Architecture >